

Clase teórica 10

**Sintaxis y Semántica
de la Lógica de Hoare**

Sensatez de H y H^*

1. Repaso. Lenguajes de la Lógica de Hoare

1. Lenguaje de programación

- Instrucciones:

$S :: \text{skip} \mid x := e \mid S_1 ; S_2 \mid \text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi} \mid \text{while } B \text{ do } S_1 \text{ od}$

- Expresiones de tipo entero:

$e :: n \mid x \mid e_1 + e_2 \mid e_1 - e_2 \mid e_1 \cdot e_2 \mid \dots$

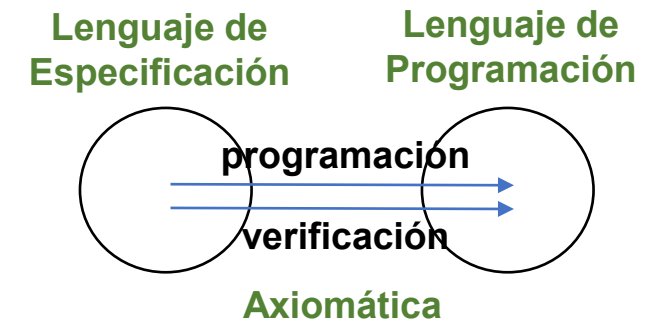
n es una constante entera. x es una variable entera.

- Expresiones de tipo booleano:

$B :: \text{true} \mid \text{false} \mid e_1 = e_2 \mid e_1 < e_2 \mid \dots \mid \neg B_1 \mid B_1 \vee B_2 \mid B_1 \wedge B_2 \mid \dots$

2. Lenguaje de especificación (lógica de predicados):

$p :: \text{true} \mid \text{false} \mid e_1 = e_2 \mid e_1 < e_2 \mid \dots \mid \neg p \mid p_1 \vee p_2 \mid \dots \mid \exists x: p \mid \forall x: p$



1. Repaso. Axiomática para correctitud parcial - MÉTODO H

1. Axioma del skip (SKIP)

$$\{p\} \text{ skip } \{p\}$$

2. Axioma de la asignación (ASI)

$$\{p[x|e]\} x := e \{p\}$$

3. Regla de la secuencia (SEC)

$$\frac{\{p\} S_1 \{r\}, \{r\} S_2 \{q\}}{\{p\} S_1 ; S_2 \{q\}}$$

4. Regla del condicional (COND)

$$\frac{\{p \wedge B\} S_1 \{q\}, \{p \wedge \neg B\} S_2 \{q\}}{\{p\} \text{ if } B \text{ then } S_1 \text{ else } S_2 \text{ fi } \{q\}}$$

5. Regla de la repetición (REP)

No asegura que el while termina

$$\frac{\{p \wedge B\} S \{p\}}{\{p\} \text{ while } B \text{ do } S \text{ od } \{p \wedge \neg B\}}$$

6. Regla de consecuencia (CONS)

$$\frac{p \rightarrow p_1, \{p_1\} S \{q_1\}, q_1 \rightarrow q}{\{p\} S \{q\}}$$

1. Repaso. Axiomática para correctitud total. MÉTODO H*

$$\text{Regla REP*}: \frac{\langle p \wedge B \rangle S \langle p \rangle, \quad \langle p \wedge B \wedge (t = Z) \rangle S \langle t < Z \rangle, \quad p \rightarrow t \geq 0}{\langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle}$$

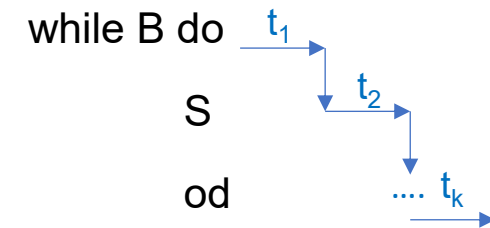
Asegura que el while termina

Se agregan dos premisas a REP, y se reemplazan los símbolos $\{ \}$ por $\langle \rangle$.

Por la segunda premisa, **t se decrementa en cada iteración.**

Z es una variable lógica, no aparece en p, B, t ni S.

El objetivo de Z es fijar el valor de t antes de la ejecución de S.



La función t representa el máximo número de iteraciones del while

Por la tercera premisa, **t se mantiene ≥ 0 a lo largo de todo el while.**

Así, el while debe terminar, porque en $\mathcal{N}$ (números naturales) **no hay cadenas descendentes infinitas**

$(\mathcal{N}, <)$ es un ejemplo de relación de orden bien fundada (todo subconjunto tiene minimal).

1. Repaso. Separación de la prueba del while en dos

- Pruebas distintas:
 - Correctitud parcial: **inducción** (invariante que vale al comienzo y luego de cada iteración).
 - Terminación: **relación de orden bien fundada** (variante entero ≥ 0 que se decrementa tras cada iteración).
- Esquema recomendable:

$$\begin{array}{ll} \text{(a) } \{p\} \text{ while } B \text{ do } S \text{ od } \{q\} & \text{Propiedad safety} \\ \text{(b) } \langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle \text{true} \rangle & \text{Propiedad liveness} \end{array}$$

$$\langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle q \rangle$$

- Ejemplo: $\langle x > 0 \rangle S_{\text{fac}} :: a := 1 ; y := 1 ; \text{ while } a < x \text{ do } a := a + 1 ; y := y \cdot a \text{ od } \langle y = x! \rangle$

Para probar $\{x > 0\} S_{\text{fac}} \{y = x!\}$ se puede utilizar el invariante $y = a! \wedge a \leq x$

Para probar $\langle x > 0 \rangle S_{\text{fac}} \langle \text{true} \rangle$ se puede utilizar el invariante más simple: $a \leq x$

2. Repaso. Pruebas sintácticas y semánticas

Programa S_{swap} que permuta x con y :

$$\begin{array}{l} S_{\text{swap}} :: \\ z := x ; \\ x := y ; \\ y := z \end{array}$$

Prueba de $\{x = X \wedge y = Y\} S_{\text{swap}} \{y = X \wedge x = Y\}$

1) Prueba **semántica**, usando la semántica del lenguaje de programación.

<i>variables</i>	<i>programa</i>
1) $x = X, y = Y$	$z := x ; x := y ; y := z$
2) $x = X, y = Y, z = X$	$x := y ; y := z$
3) $x = Y, y = Y, z = X$	$y := z$
4) $x = Y, y = X, z = X$	

2) Prueba **sintáctica**, usando los axiomas y reglas de la Lógica de Hoare.

- | | |
|---|-----------|
| 1) $\{z = X \wedge x = Y\} y := z \{y = X \wedge x = Y\}$ | ASI |
| 2) $\{z = X \wedge y = Y\} x := y \{z = X \wedge x = Y\}$ | ASI |
| 3) $\{x = X \wedge y = Y\} z := x \{z = X \wedge y = Y\}$ | ASI |
| 4) $\{x = X \wedge y = Y\} z := x ; x := y ; y := z \{y = X \wedge x = Y\}$ | SEC 1,2,3 |

2. Repaso. Pruebas sintácticas y semánticas (continuación)

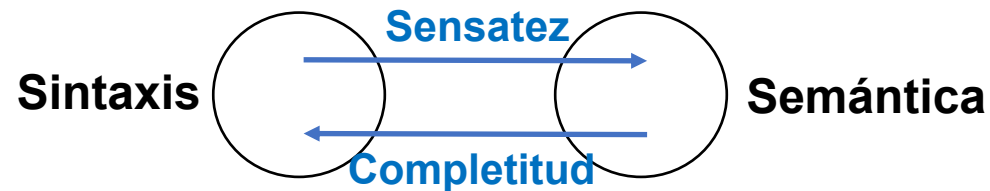
Prueba sintáctica de S_{swap} usando los axiomas y reglas de la Lógica de Hoare:

- | | |
|---|-----------|
| 1) $\{z = X \wedge x = Y\} y := z \{y = X \wedge x = Y\}$ | ASI |
| 2) $\{z = X \wedge y = Y\} x := y \{z = X \wedge x = Y\}$ | ASI |
| 3) $\{x = X \wedge y = Y\} z := x \{z = X \wedge y = Y\}$ | ASI |
| 4) $\{x = X \wedge y = Y\} z := x ; x := y ; y := z \{y = X \wedge x = Y\}$ | SEC 1,2,3 |

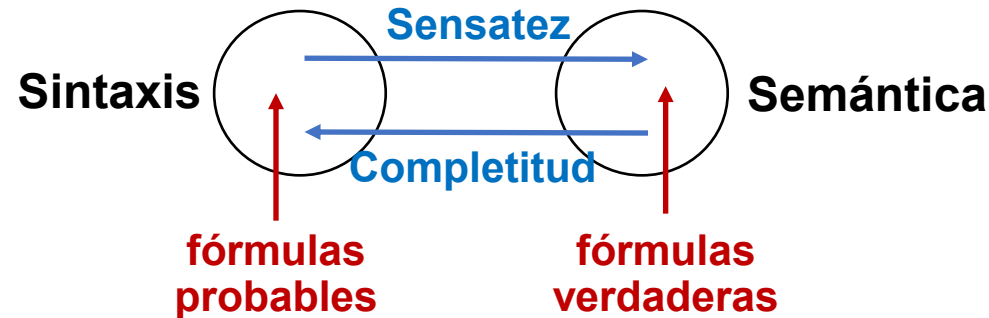
Por la **sensatez** de la lógica de Hoare, $\{x = X \wedge y = Y\} z := x ; x := y ; y := z \{y = X \wedge x = Y\}$ es verdadera

Obligatorio: un método axiomático debe ser **sensato**: no puede probar fórmulas falsas.

Deseable: un método axiomático debe ser **completo**: debe poder probar todas las fórmulas verdaderas.



3. Sensatez y completitud de H y H*



Sensatez de H

No puede pasar que se demuestre en H, por ejemplo, $\{x = 0\} x := x + 1 \{x = 2\}$

Completitud de H

No puede pasar que no se pueda demostrar en H, por ejemplo, $\{x = 0\} y := x \{y = 0\}$

Sensatez de H*

No puede pasar que se demuestre en H*, por ejemplo, $\langle x = 0 \rangle \text{ while } x = 0 \text{ do skip od } \langle \text{true} \rangle$

Completitud de H*

No puede pasar que no se pueda demostrar en H*, por ejemplo, $\langle x = 0 \rangle \text{ while } x < 10 \text{ do } x := x + 1 \text{ od } \langle x = 10 \rangle$

3. Sensatez y completitud de H y H* (continuación)

$\vdash_H \{p\} S \{q\}$ expresa que en H se puede demostrar $\{p\} S \{q\}$
 $\models \{p\} S \{q\}$ expresa que se cumple **semánticamente** $\{p\} S \{q\}$

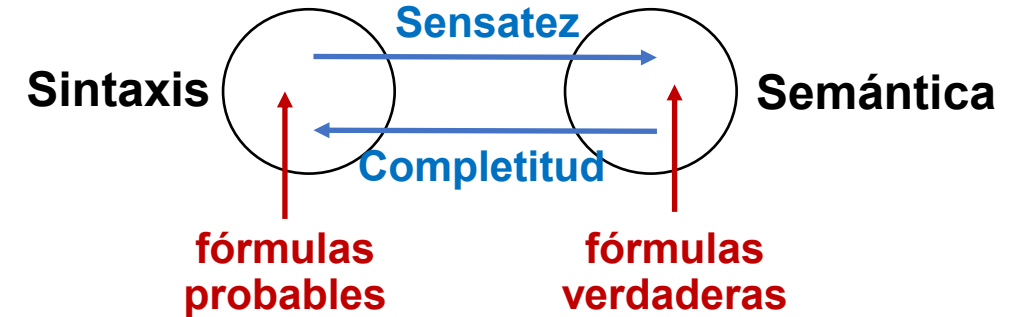
$\vdash_{H^*} \langle p \rangle S \langle q \rangle$ expresa que en H* se puede demostrar $\langle p \rangle S \langle q \rangle$
 $\models \langle p \rangle S \langle q \rangle$ expresa que se cumple **semánticamente** $\langle p \rangle S \langle q \rangle$

Se prueba que:

1. $\vdash_H \{p\} S \{q\} \rightarrow \models \{p\} S \{q\}$ (sensatez de H)
2. $\models \{p\} S \{q\} \rightarrow \vdash_H \{p\} S \{q\}$ (completitud de H)
1. $\vdash_{H^*} \langle p \rangle S \langle q \rangle \rightarrow \models \langle p \rangle S \langle q \rangle$ (sensatez de H*)
2. $\models \langle p \rangle S \langle q \rangle \rightarrow \vdash_{H^*} \langle p \rangle S \langle q \rangle$ (completitud de H*)

En palabras:

- Las fórmulas que demuestran H y H* son **verdaderas** (sensatez).
- Todas las fórmulas verdaderas **se pueden demostrar** en H y H* (completitud).



4. Inducción matemática

- Sea P una propiedad que se quiere probar en $\mathcal{N}$ (los números naturales).

- Si:

- 1) Se cumple: $P(0)$

- 2) Para todo k se cumple: $P(k) \rightarrow P(k + 1)$

Entonces:

Para todo $n = 0, 1, 2, \dots$, se cumple: $P(n)$

(1) es la **base inductiva**

(2) es el **paso inductivo** ($P(k)$ es la **hipótesis inductiva**)

- Variante fuerte de la inducción matemática

Paso inductivo: Si $P(i) \wedge P(i + 1) \wedge \dots \wedge P(k - 1) \wedge P(k)$ para algún i
entonces $P(k + 1)$

Ejemplo: para todo n se cumple:

$$0 + 1 + 2 + \dots + n = n(n + 1)/2$$

Prueba

1) $P(0)$: $0 = 0(0 + 1)/2$

2) $P(k)$: $0 + \dots + k = k(k + 1)/2$

Veamos que también se cumple $P(k + 1)$

$$\begin{aligned} &0 + \dots + k + (k + 1) \\ &= k(k + 1)/2 + (k + 1) - \text{hipótesis inductiva} \\ &= (k + 1)(k/2 + 1) \\ &= (k + 1)(k + 2)/2 \end{aligned}$$

4. Inducción estructural

- Más general que la inducción matemática.
- Util para definir conjuntos y demostrar sus propiedades.
- Se aplica a cualquier dominio (números naturales, expresiones aritméticas, árboles, programas, etc).
- Puede haber varias bases inductivas y varios pasos inductivos.
- **Ejemplo**

Definir por inducción estructural las expresiones aritméticas con símbolos de $\{ 0, 1, x, +, \cdot, (,) \}$

(a) bases inductivas: 0, 1, x, son expresiones aritméticas.

(b) pasos inductivos: si e y f son expresiones aritméticas, también lo son $(e + f)$ y $(e \cdot f)$.

Ejemplo: probar $C(e) = 1 + O(e)$

C = cantidad de 0, 1 y x

O = cantidad de operadores + y $\cdot$.

Bases inductivas

$$C(0) = 1 + O(0)$$

$$C(1) = 1 + O(1)$$

$$C(x) = 1 + O(x)$$

Paso inductivo con respecto al operador +

Supongamos: $C(e) = 1 + O(e)$ y $C(f) = 1 + O(f)$

Luego:

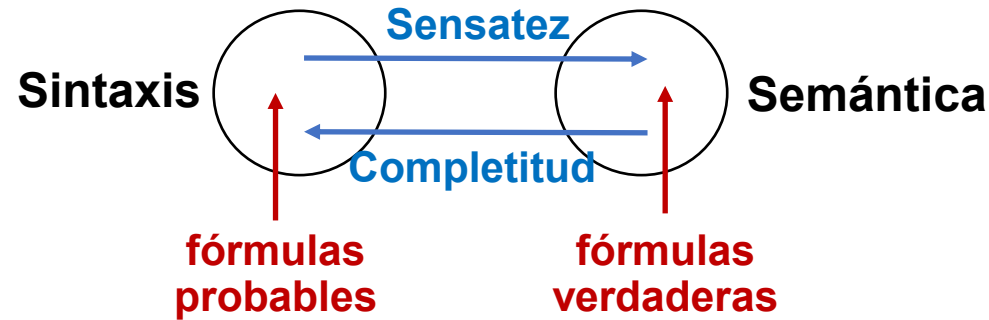
$$C(e + f) = C(e) + C(f) = 1 + O(e) + 1 + O(f) - \text{Hip.}$$

$$\text{Y como: } O(e + f) = O(e) + 1 + O(f)$$

$$\text{Entonces: } C(e + f) = 1 + O(e + f)$$

Ejercicio. Hacer el paso inductivo con respecto al operador $\cdot$.

5. Semántica del lenguaje de programación



- Para probar las propiedades de sensatez y completitud de H y H^* , primero tenemos que definir **formalmente** la **semántica del lenguaje de programación**.
- **Idea general.** Dados un programa S y un estado inicial σ , a dicha **instancia** o **configuración inicial** (S, σ) se le asocia una **computación** $\pi(S, \sigma)$, que es la secuencia de instancias de la ejecución de S a partir de σ .

P.ej.: $(x := 0 ; y := 1 ; z := 2, \sigma) \rightarrow (y := 1 ; z := 2, \sigma[x|0]) \rightarrow (z := 2, \sigma[x|0][y|1]) \rightarrow (E, \sigma[x|0][y|1][z|2])$.

- **Notación.** $\text{val}(\pi(S, \sigma))$ es el estado final de $\pi(S, \sigma)$. Si $\pi(S, \sigma)$ es infinita, $\text{val}(\pi(S, \sigma)) = \perp$.

P.ej.: $\text{val}(\pi(\text{while true do skip od}, \sigma)) = \perp$, y $\text{val}(\pi(x := 10 ; \text{while } x > 0 \text{ do } x := x - 1 \text{ od}, \sigma)) = \sigma[x|0]$.

5. Semántica del lenguaje de programación (continuación)

- El lenguaje de programación se define por **inducción estructural**, mediante una relación $\rightarrow$
 1. $(\text{skip}, \sigma) \rightarrow (E, \sigma)$
El **skip** se consume en un paso (es atómico) y no modifica el estado inicial. E es la continuación sintáctica vacía.
 2. $(x := e, \sigma) \rightarrow (E, \sigma[x|\sigma(e)])$
La **asignación** $x := e$ es atómica y el estado final es como el inicial salvo que el valor de x es el de la expresión e según σ .
 3. Para toda instrucción T, si $(S, \sigma) \rightarrow (S', \sigma')$, entonces $(S ; T, \sigma) \rightarrow (S' ; T, \sigma')$
La **secuencia** $S;T$ se ejecuta de izquierda a derecha. Consumido S, si no diverge (si no loopea), se ejecuta T.
 4. $(\text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi}, \sigma) \rightarrow (S_1, \sigma)$, si se cumple $\sigma(B) = \text{verdadero}$.
 $(\text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi}, \sigma) \rightarrow (S_2, \sigma)$, si se cumple $\sigma(B) = \text{falso}$.
La evaluación de la expresión B es atómica y no modifica el estado inicial. Su evaluación establece si se ejecuta S_1 o S_2 .
 5. $(\text{while } B \text{ do } S \text{ od}, \sigma) \rightarrow (S; \text{while } B \text{ do } S \text{ od}, \sigma)$, si se cumple $\sigma(B) = \text{verdadero}$.
 $(\text{while } B \text{ do } S \text{ od}, \sigma) \rightarrow (E, \sigma)$, si se cumple $\sigma(B) = \text{falso}$.
La evaluación de B es atómica y no modifica el estado inicial. Su evaluación establece si se ejecuta S o termina la repetición.

- **Nota**

Las expresiones $\sigma(e)$ y $\sigma(B)$ son abreviaciones, porque un estado σ se define sobre variables, no sobre expresiones.

Por ejemplo, si e tiene la forma $x + 8$, entonces **$\sigma(e)$ es en realidad $\sigma(x) + \sigma(8)$** .

- **Definición por extensión de la semántica de las instrucciones**

De la definición por **comprensión** (o **inductiva**) de las instrucciones se deriva su definición por **extensión**.

Por ejemplo, en el caso de la **secuencia**, una computación $\pi(S_1; S_2, \sigma)$ tiene tres formas posibles:

1. Una computación infinita $(S_1 ; S_2, \sigma_0) \rightarrow (T_1 ; S_2, \sigma_1) \rightarrow (T_2 ; S_2, \sigma_2) \rightarrow \dots$, cuando S_1 diverge a partir de σ_0 .
2. Una computación infinita $(S_1 ; S_2, \sigma_0) \rightarrow \dots \rightarrow (S_2, \sigma_1) \rightarrow (T_1, \sigma_2) \rightarrow (T_2, \sigma_3) \rightarrow \dots$, cuando S_1 termina a partir de σ_0 y S_2 diverge a partir de σ_1 .
3. Una computación finita $(S_1 ; S_2, \sigma_0) \rightarrow \dots \rightarrow (S_2, \sigma_1) \rightarrow \dots \rightarrow (E, \sigma_2)$, cuando S_1 termina a partir de σ_0 y S_2 termina a partir de σ_1 .

De modo similar se pueden obtener las formas de las computaciones de las otras instrucciones.

Ejercicio. Obtener las formas de las computaciones de las otras instrucciones.

6. Prueba de la sensatez del método H

- Se cumple, para todo programa S y toda especificación (p, q) , que: $\vdash_H \{p\} S \{q\} \rightarrow \models \{p\} S \{q\}$.
- Se prueba por **inducción matemática (fuerte)**, considerando la **longitud de la prueba** (1 o más pasos):
 - (a) Base inductiva: los **axiomas** son verdaderos (pruebas de tamaño 1).
 - (b) Paso inductivo: las **reglas** son sensatas: preservan la verdad de las premisas (pruebas de tamaño ≥ 2).
- **Ejemplo de (a). Prueba de que el axioma SKIP es verdadero.**

Dado $\vdash \{p\} \text{ skip } \{p\}$, hay que probar $\models \{p\} \text{ skip } \{p\}$.

Axioma del skip (SKIP)
 $\{p\} \text{ skip } \{p\}$

Sea $\vdash \{p\} \text{ skip } \{p\}$.

Sea $\sigma \models p$.

La semántica del skip es: $(\text{skip}, \sigma) \rightarrow (E, \sigma)$.

Así, luego de un skip queda $\sigma \models p$.

Por lo tanto, $\models \{p\} \text{ skip } \{p\}$.

Ejercicio. Probar que el axioma ASI es verdadero.

6. Prueba de la sensatez del método H (continuación)

- Ejemplo de (b). Prueba de que la regla SEC es sensata (preserva la verdad de las premisas):

Dado $\vdash \{p\} S_1 ; S_2 \{q\}$, hay que probar $\models \{p\} S_1 ; S_2 \{q\}$.

Sea $\vdash \{p\} S_1 ; S_2 \{q\}$.

Se cumple $\vdash \{p\} S_1 \{r\}$ y $\vdash \{r\} S_2 \{q\}$, para algún r . ¿Por qué?

Las pruebas $\vdash \{p\} S_1 \{r\}$ y $\vdash \{r\} S_2 \{q\}$ son más cortas que la prueba $\vdash \{p\} S_1 ; S_2 \{q\}$.

Por lo tanto, se puede aplicar inducción.

Por hipótesis inductiva: $\models \{p\} S_1 \{r\}$ y $\models \{r\} S_2 \{q\}$. ¿Por qué?

Sea $\sigma_1 \models p$.

La semántica de la secuencia es: $(S_1 ; S_2, \sigma_1) \rightarrow \dots \rightarrow (S_2, \sigma_2) \rightarrow \dots \rightarrow (E, \sigma_3)$

Así, como $\models \{p\} S_1 \{r\}$ y $\models \{r\} S_2 \{q\}$, luego de $S_1 ; S_2$ queda $\sigma_3 \models q$.

Por lo tanto, $\models \{p\} S_1 ; S_2 \{q\}$.

Regla de la secuencia (SEC)

$\{p\} S_1 \{r\}, \{r\} S_2 \{q\}$

$\{p\} S_1 ; S_2 \{q\}$

Ejercicio. Probar la sensatez de las otras reglas.

6. Prueba de la sensatez del método H*

- Sólo hay que probar que la regla REP* es sensata:

$$\vdash_{H^*} \langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle \rightarrow \models \langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle$$

Regla REP*
$\frac{\langle p \wedge B \rangle S \langle p \rangle, \langle p \wedge B \wedge t = Z \rangle S \langle t < Z \rangle, p \rightarrow t \geq 0}{\langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle}$

Sea $\vdash \langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle$.

Se cumple $\vdash \langle p \wedge B \rangle S \langle p \rangle$, $\vdash \langle p \wedge B \wedge t = Z \rangle S \langle t < Z \rangle$, $p \rightarrow t \geq 0$. ¿Por qué?

$\vdash \langle p \wedge B \rangle S \langle p \rangle$ y $\vdash \langle p \wedge B \wedge t = Z \rangle S \langle t < Z \rangle$ son pruebas más cortas que $\vdash \langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle$.

Por lo tanto, se puede aplicar inducción.

Por hipótesis inductiva: $\models \langle p \wedge B \rangle S \langle p \rangle$, $\models \langle p \wedge B \wedge t = Z \rangle S \langle t < Z \rangle$, $p \rightarrow t \geq 0$. ¿Por qué?

Sea $\sigma_0 \models p$. Hay que probar que el while termina en un estado $\sigma_k \models \langle p \wedge \neg B \rangle$.

Supongamos que no, que el while loopea desde σ_0 .

Así: $(\text{while } B \text{ do } S \text{ od}, \sigma_0) \rightarrow (\text{while } B \text{ do } S \text{ od}, \sigma_1) \rightarrow (\text{while } B \text{ do } S \text{ od}, \sigma_2) \dots$,
y por la hipótesis inductiva, **siempre vale p , $t \geq 0$, y t decrece después de cada iteración.**

Pero esto no puede ocurrir, porque si no habría una cadena descendente infinita de números naturales.

Por lo tanto, el while termina, y lo hace en algún estado **$\sigma_k \models \langle p \wedge \neg B \rangle$.**

Anexo

Sensatez total

- La sensatez de H es **total**, es decir, se cumple **independientemente** del dominio semántico considerado (números naturales, números reales, etc).
- Por ejemplo, en el programa de división presentado antes, la prueba también aplica si las variables son de tipo real:

$\{x \geq 0 \wedge y > 0\} S_{\text{div}} \{x = y \cdot c + r \wedge r < y \wedge r \geq 0\}$, con:

$S_{\text{div}} :: c := 0 ; r := x ; \text{while } r \geq y \text{ do } r := r - y ; c := c + 1 \text{ od}$

- La idea es que H incluya todos los axiomas del dominio semántico considerado.
- En cambio, la sensatez de H* es **aritmética**. El dominio semántico debe **extenderse** con el de los **números naturales**, para definir el variante t.

Sensatez aritmética

- Sea el siguiente programa en el que las variables x y ϵ son de tipo **real**, inicialmente mayores que 0:

```
S :: while  $x > \epsilon$   
do  
 $x := x / 2$   
od
```

- Se cumple: $\langle x = X \wedge X > 0 \wedge \epsilon > 0 \rangle S \langle \text{true} \rangle$.
- Los valores positivos de x forman una secuencia decreciente estricta.
- También en este caso se puede usar la regla REP*, con un variante t del dominio de los números naturales.
- Un posible t es:

$$t = \text{if } x > \epsilon \text{ then } \lceil \log_2 X/\epsilon \rceil - \log_2 X/x \text{ else } 0 \text{ fi}$$

Semántica funcional (o denotacional) de las expresiones y predicados

- Expresiones enteras

$$e :: n \mid x \mid (e_1 + e_2) \mid (e_1 - e_2) \mid (e_1 \cdot e_2) \mid \dots$$

n es una constante entera. x es una variable entera. P.ej: $e = (5 + (x - 1))$

- Se utiliza una **función semántica V** , que asigna a una **expresión entera e** y un **estado σ** un **número entero n**

$$V : \text{lexp} \rightarrow (\Sigma \rightarrow \mathbb{Z})$$

lexp es el conjunto de las expresiones enteras

- Definición inductiva

1. $V(n)(\sigma) = n$, tal que n es la representación del número entero n (p.ej., el símbolo 1 representa el número 1)
2. $V(x)(\sigma) = \sigma(x)$
3. $V((e_1 + e_2))(\sigma) = V(e_1)(\sigma) + V(e_2)(\sigma)$
4. $V((e_1 - e_2))(\sigma) = V(e_1)(\sigma) - V(e_2)(\sigma)$

Etc.

- Expresiones booleanas

$B :: \text{true} \mid \text{false} \mid (e_1 = e_2) \mid (e_1 < e_2) \mid \dots \mid \neg B_1 \mid (B_1 \vee B_2) \mid (B_1 \wedge B_2) \mid \dots$

true y false son las constantes booleanas. Las e_i son expresiones enteras. P.ej: $B = ((x = 1) \vee (x = 2))$

- Se utiliza una **función semántica W**, que asigna a una **expresión booleana B** y un **estado σ** un **valor V o F**

$W : B_{\text{exp}} \rightarrow (\Sigma \rightarrow \{V, F\})$

B_{exp} es el conjunto de las expresiones booleanas

- Definición inductiva

1. $W(\text{true})(\sigma) = V$ y $W(\text{false})(\sigma) = F$ (p.ej., la constante true representa el valor de verdad V)

2. $W((e_1 = e_2))(\sigma) = (V(e_1)(\sigma) = V(e_2)(\sigma))$

3. $W((e_1 < e_2))(\sigma) = (V(e_1)(\sigma) < V(e_2)(\sigma))$

4. $W(\neg B_1)(\sigma) = \neg W(B_1)(\sigma)$

5. $W((B_1 \vee B_2))(\sigma) = W(B_1)(\sigma) \vee W(B_2)(\sigma)$

Etc.

- Predicados

$$p :: \text{true} \mid \text{false} \mid (e_1 = e_2) \mid (e_1 < e_2) \mid \dots \mid \neg p \mid (p_1 \vee p_2) \mid \dots \mid \exists x: p \mid \forall x: p$$

true y false son las constantes. x es una variable entera. Las e_i son expresiones enteras. P.ej: $\exists x: \neg(x = 1)$

- Se utiliza una **función semántica** T , que asigna a un **predicado** p y un **estado** σ un **valor** V o F

$$T : \text{Pred} \rightarrow (\Sigma \rightarrow \{V, F\})$$

Pred es el conjunto de los predicados

- Definición inductiva

1. $T(\text{true})(\sigma) = V$ y $T(\text{false})(\sigma) = F$ (p.ej., la constante true representa el valor de verdad V)
2. $T((e_1 = e_2))(\sigma) = (T(e_1)(\sigma) = T(e_2)(\sigma))$. Y lo mismo con los otros operadores ($<$, $>$, etc.)
3. $T(\neg p_1)(\sigma) = \neg T(p_1)(\sigma)$. Y lo mismo con los otros operadores ($\wedge$, $\vee$, etc.)
4. $T(\exists x: p_1)(\sigma) = V$ sii existe algún número entero n tal que $T(p_1)(\sigma[x|n]) = V$
5. $T(\forall x: p_1)(\sigma) = V$ sii para todo número entero n se cumple $T(p_1)(\sigma[x|n]) = V$

Semántica funcional (o denotacional) de las instrucciones

- Se utiliza una función semántica M , que asigna a un programa S y un estado σ otro estado σ .

$$M : L_w \rightarrow (\Sigma \rightarrow \Sigma)$$

L_w es el lenguaje de programación

- Definición inductiva

- $M(\text{skip})(\sigma) = \sigma$
- $M(x := e)(\sigma) = \sigma[x|V(e)(\sigma)]$
- $M(S_1; S_2)(\sigma) = M(S_2)(M(S_1)(\sigma))$
- $M(\text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi})(\sigma) = M(S_1)(\sigma), \text{ si } W(B)(\sigma) = V$
 $= M(S_2)(\sigma), \text{ si } W(B)(\sigma) = F$
- $M(\text{while } B \text{ do } S \text{ od})(\sigma) = M(\text{while } B \text{ do } S \text{ od})(M(S)(\sigma)), \text{ si } W(B)(\sigma) = V$
 $= \sigma, \text{ si } W(B)(\sigma) = F$
con $M(S)(\sigma) = \perp$ si S diverge, y $M(S)(\perp) = \perp$ para todo S

- La especificación así planteada está incompleta, falta encontrar la solución de $M(\text{while } B \text{ do } S \text{ od})(\sigma)$

Una manera es calculando el límite de una secuencia de funciones conocidas como *aproximaciones*, que representan las sucesivas iteraciones del *while*.

Semántica operacional vs semántica funcional (o denotacional)

Semántica operacional

- Definida con computaciones, configuraciones, estados, etc., de una máquina.
- Muy intuitiva.
- Sirve como guía de implementación (el nivel de abstracción elegido es muy importante).

Semántica funcional (o denotacional)

- Definida con conjuntos, funciones, funcionales, etc. Se relacionan estructuras sintácticas con objetos matemáticos. Toda estructura **denota** un valor.
- No tiene el riesgo del nivel de abstracción.
- Su uso se dificulta cuando el lenguaje es muy complejo, sobre todo concurrente.

En definitiva, las dos semánticas son complementarias más que alternativas.

Clase práctica 10

Ejemplo 1. Computación de un programa.

- Sea el programa $S_{\text{swap}} :: z := x ; x := y ; y := z$, y el estado inicial σ_0 , con $\sigma_0(x) = 1$ y $\sigma_0(y) = 2$.
- Utilizando la relación de transición $\rightarrow$ se prueba que S_{swap} intercambia los contenidos de las variables x e y :

$$\begin{aligned}\pi(S_{\text{swap}}, \sigma_0) &= (z := x ; x := y ; y := z, \sigma_0[x|1][y|2]) \rightarrow \\ &\quad (x := y ; y := z, \sigma_0[x|1][y|2][z|1]) \rightarrow \\ &\quad (y := z, \sigma_0[y|2][z|1][x|2]) \rightarrow \\ &\quad (E, \sigma_0[z|1][x|2][y|1])\end{aligned}$$

- Quedó: $\text{val}(\pi(S_{\text{swap}}, \sigma_0)) = \sigma_1$, con $\sigma_1(x) = 2$ y $\sigma_1(y) = 1$.
- Generalizando, si $\sigma_0(x) = X$ y $\sigma_0(y) = Y$, entonces $\text{val}(\pi(S_{\text{swap}}, \sigma_0)) = \sigma_1$, con $\sigma_1(x) = Y$ y $\sigma_1(y) = X$.

En programas simples como éste, la **verificación semántica** resulta factible, pero en programas complejos se torna **prohibitiva**.

Ejemplo 2. Sensatez de la Regla de la Disyunción (OR)

La regla OR establece, dados predicados p , q , r , y un programa S :

$$\frac{\{p\} S \{q\}, \{r\} S \{q\}}{\{p \vee r\} S \{q\}}$$

Probaremos la sensatez de la regla OR por inducción sobre la longitud de la prueba. Hay que probar:

$\vdash \{p \vee r\} S \{q\} \rightarrow \models \{p \vee r\} S \{q\}$, habiendo aplicado la regla OR

- $\vdash \{p \vee r\} S \{q\}$ proviene de $\vdash \{p\} S \{q\}$ y $\vdash \{r\} S \{q\}$, aplicando la regla OR.
- Hipótesis inductiva: (1) $\models \{p\} S \{q\}$, (2) $\models \{r\} S \{q\}$.
- Veamos que se cumple $\models \{p \vee r\} S \{q\}$:
- Sea $\sigma \models p \vee r$, y supongamos que S termina desde σ en un estado σ' .
- Hay dos posibilidades, lo que completa la prueba:
 - Si $\sigma \models p$, entonces por (1) vale $\sigma' \models q$.
 - Si $\sigma \models r$, entonces por (2) vale $\sigma' \models q$.